BPAC · BANGLA PARALLEL ACCENT CORPUS

How Detectable Is Regional Accent in Read Bangla?

A step-by-step walk through every analysis in this repository — what each script measures, what method it uses, and what it found — in the order the pipeline actually runs.

51 native speakers across **8 regions** of Bangladesh each read the identical **250-sentence** script, so region and speaker vary while linguistic content is held fixed by design. The pipeline runs as five parts — **A** (descriptive EDA), **B** (ASR benchmark & alignment verification), **C** (neural embedding accent structure), **D** (classical acoustic-phonetic map), **E** (validation) — each an independent Python package with its own `config.yml`, `analyze.py` scripts, and `reports/` output. Every number below is quoted from those reports.

51	8	250	12,750	12,747
speakers	regions	shared sentences	raw utterances	utterances after cleaning
27.74h				
total audio				

The one confound that runs through everything: male participants were recorded in a sound-treated lab; female participants recorded independently through an Android app (*SUST Kotha*). Recording pipeline is therefore **perfectly confounded with gender** (lab = male, app = female) — every Part below treats that as a hard constraint, not something to explain away.

PART A · STEPS A1–A8

Corpus descriptive EDA

Eight independent analyses answering: is the corpus actually what it claims to be? Full parallel coverage, duration balance, character/word composition of the shared script, speaking rate, pitch, and audio quality — all computed identically across regions, all reading from `config.yml` rather than command-line flags.

`package bpac_eda · data data/BPAC/<region>/speaker_<gender>_<id>_<name>/sound_<NNN>.wav`

A1 Unique sentences & parallel coverage

`A1_unique_sentences/analyze.py`

Purpose. Confirms the corpus is genuinely parallel: every speaker recorded every one of the 250 sentences, and no sentence text repeats.

METHOD

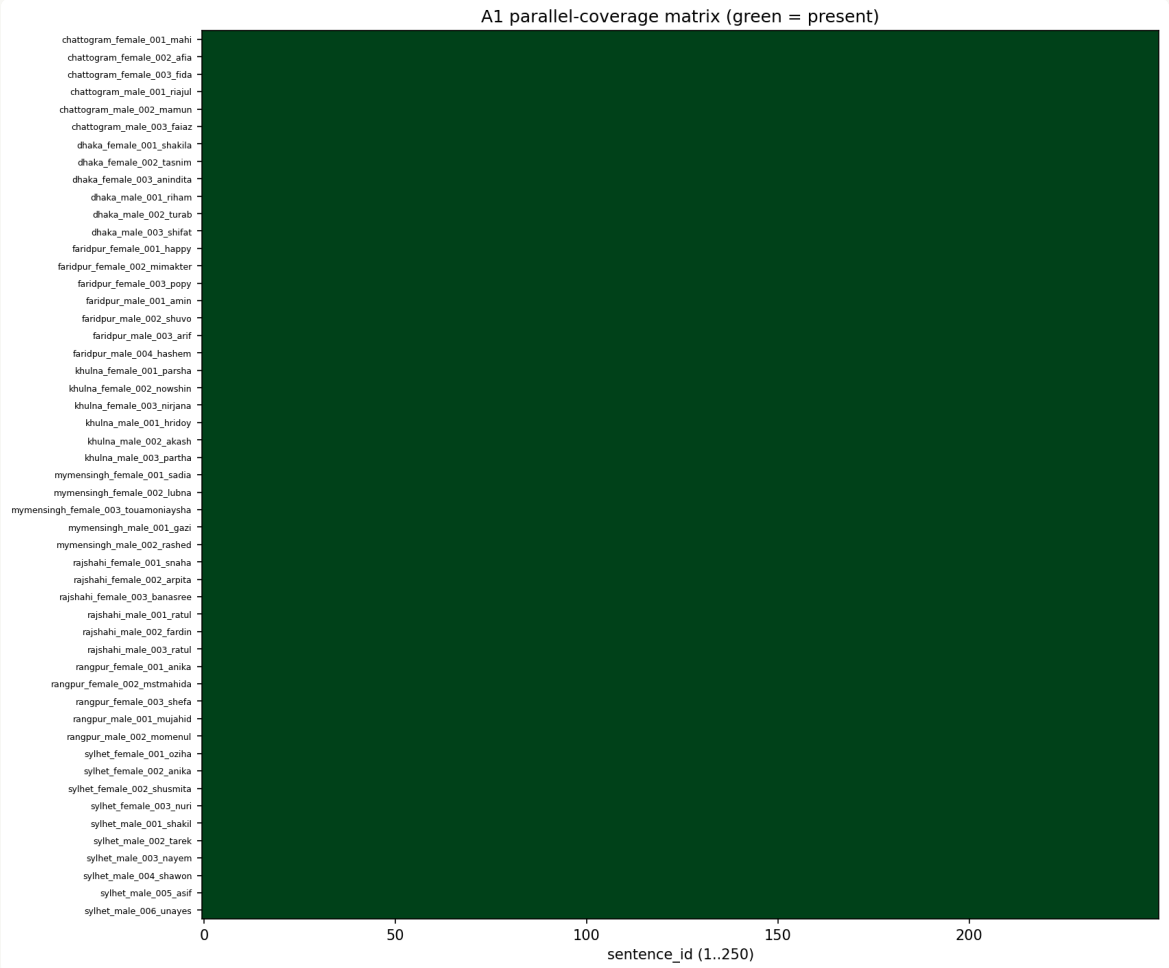
Loads the 250-sentence script (NFC-normalized), counts unique text vs. row count, builds the full utterance manifest, and forms a boolean speaker × sentence_id presence matrix (`groupby(...).size().unstack().gt(0)`) reindexed to every expected (speaker, sentence) pair. A deterministic coverage audit — no statistical test.

KEY NUMBERS

250 / 250	51 / 51	0	0
unique sentence texts	speakers complete	missing cells	duplicate texts

Result: `fully_parallel = True` — every speaker has every sentence and there are no duplicate sentence texts.

FIGURE



Speaker × sentence-id (1–250) presence matrix — green = present. Full green field confirms complete coverage.

A2 Audio length

`A2_audio_length/analyze.py`

Purpose. Total and distributional audio duration by region, speaker, and gender, to check for size/duration imbalance that could confound region-vs-speaker comparisons.

METHOD

Mode is `raw`: duration read from the file header via `soundfile.info` (not silence-trimmed). Aggregated as sum/mean/count grouped by region, gender, region×gender, and speaker.

KEY NUMBERS

27h 44m 29s	7.833s	23.0–48.3 min
total audio	mean utterance	per-speaker range

Sylhet contributes the most total audio (297.2 min) simply because it has the most speakers (10); per-utterance means are close across regions (7.1–8.4 s). The largest gender gap within a region is Sylhet: 192.2 male-minutes vs. 105.0 female-minutes.

REGION	TOTAL (MIN)	MEAN (S)
Sylhet	297.18	7.13
Faridpur	231.14	7.92
Dhaka	210.20	8.41
Chattogram	206.72	8.27
Khulna	200.29	8.01
Rajshahi	190.30	7.61
Mymensingh	167.04	8.02
Rangpur	161.62	7.76

A3 Utterance counts

`A3_utterance_count/analyze.py`

Purpose. Counts utterances per region/speaker/gender and checks every speaker against the expected 250-sentence quota.

METHOD

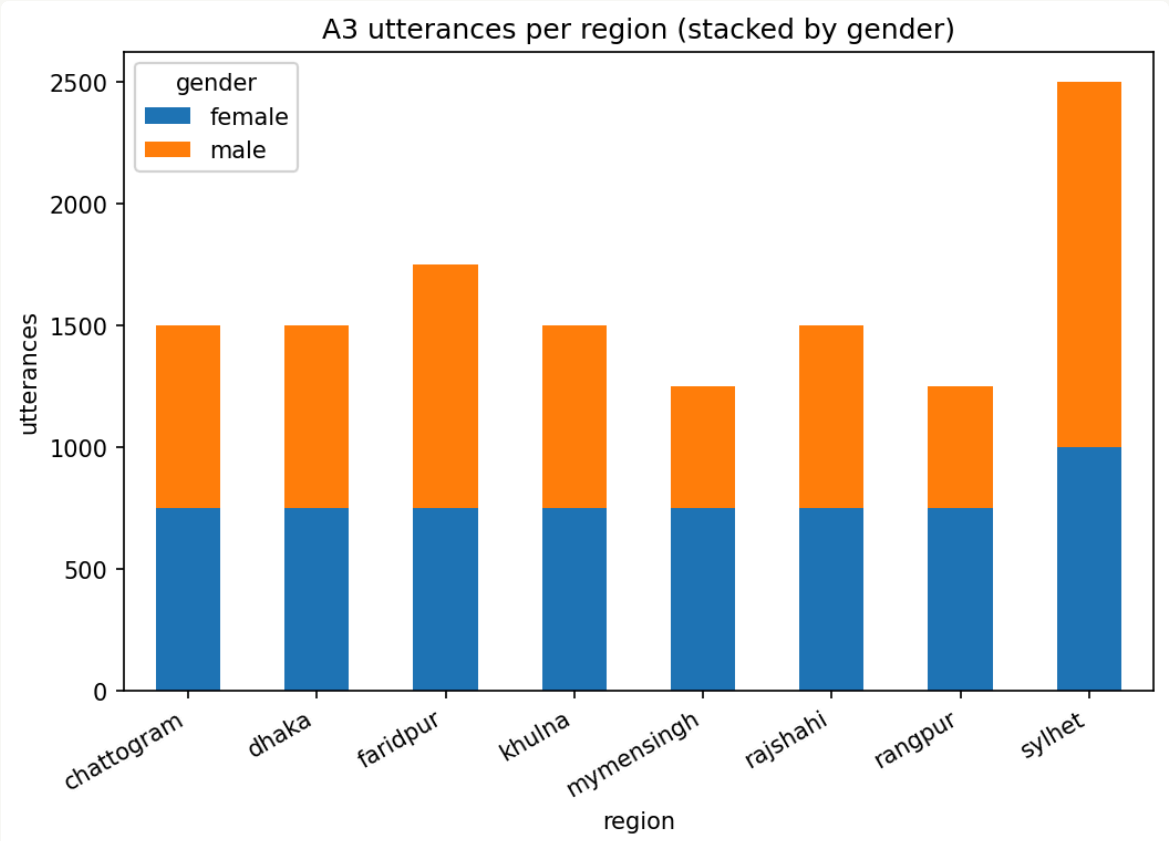
Pure `groupby().size()` counting; `delta_vs_expected = n_utt − 250` per speaker. Descriptive only.

KEY NUMBERS

12,750	0	0
total utterances	speakers under quota	speakers over quota

Speakers per region range from 5 (Mymensingh, Rangpur) to 10 (Sylhet); every speaker's delta against the 250-sentence quota is exactly zero.

FIGURE



Utterances per region, stacked by gender.

A4 Character count (vowel / consonant)

`A4_character_count/analyze.py`

Purpose. Characterizes the Bangla script composition of the shared 250-sentence set — identical across all regions since the sentences are shared.

METHOD

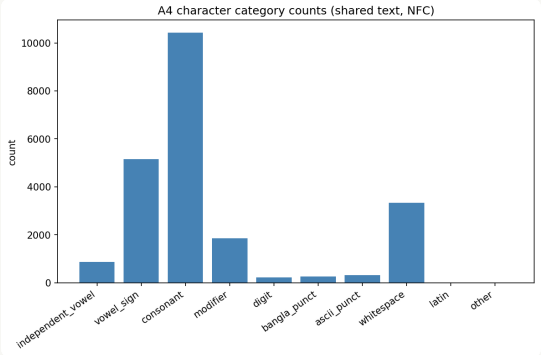
NFC-normalizes the concatenated script text, then classifies every non-space character at the code-point level against hard-coded Unicode Bengali-block ranges (independent vowel, vowel sign, consonant, modifier, digit, punctuation, whitespace, latin, other). Vowels = independent vowels + vowel signs; consonants = consonant letters. Grapheme-cluster count computed separately via Unicode `\X` clusters.

KEY NUMBERS

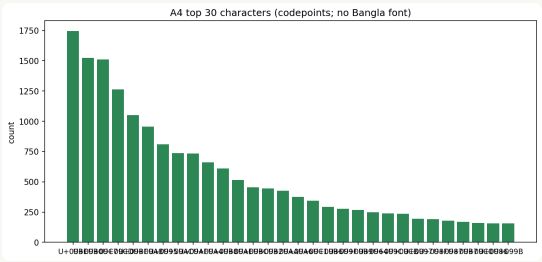
19,113	10,844	0.578	81
non-space code points	grapheme clusters	vowel / consonant ratio	unique characters

CATEGORY	COUNT	% OF NON-SPACE
Consonant	10,435	54.60%
Vowel sign (kar)	5,152	26.96%
Whitespace	3,341	17.48%
Modifier	1,860	9.73%
Independent vowel	874	4.57%
ASCII punctuation	320	1.67%
Bangla punctuation	250	1.31%
Digit	221	1.16%

No Bangla-capable font was installed at render time, so the figures below fall back to code-point / rank labels — exact glyphs live in the CSVs.



Character-category counts across the shared script.



Top 30 characters by frequency (labeled by code point).

A5 Word count & frequency

A5_word_count/analyze.py

Purpose. Lexical statistics of the shared script — token count, vocabulary size, type-token ratio, Zipf behavior.

METHOD

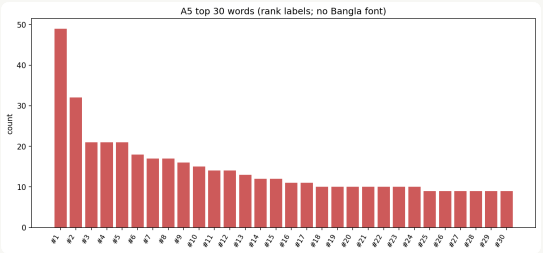
NFC-normalize, strip Bangla danda/punctuation, whitespace-tokenize (not a morphological Bangla-aware tokenizer). Frequency table ranked by count; type-token ratio = unique types ÷ total tokens; hapax legomena = words occurring exactly once.

KEY NUMBERS

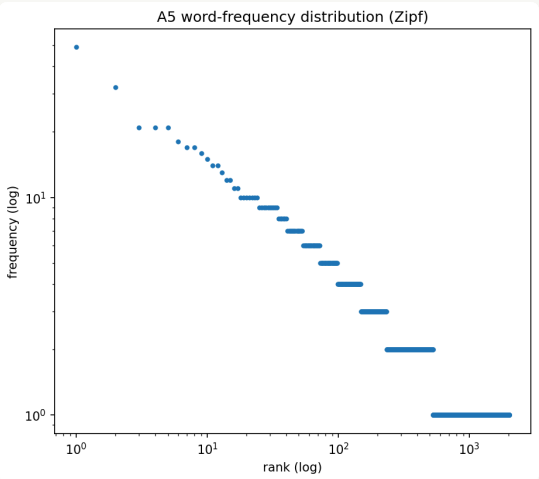
3,385	2,012	0.594	1,485
total tokens	unique types	type-token ratio	hapax legomena

13.54
words / sentence (mean)

Most frequent words: à¦“ (49×, 1.45%), à¦•à¦°à§‡ (32×), à¦□à¦‡, à¦□, à¦□à¦¬à¦, (21× each).



Top 30 words by frequency.



Log-log rank vs. frequency (Zipf) distribution.

A6 Speaking rate & pitch — energy-trim baseline

A6_speaking_rate_pitch/analyze.py

Purpose. Per-utterance speaking rate (words/min, syllables/s) and pitch (F0), aggregated by region × gender — an energy-based silence-trim baseline that A7 later corrects.

METHOD

Speech-only duration via `librosa.effects.trim(top_db=30)` (an energy-threshold trim on the whole clip, not true voice-activity detection). WPM = word count ÷ duration × 60; syllables/s via a heuristic Bangla vowel-nucleus counter. F0 via Praat (`parselmouth`, per-gender floor/ceiling 75–300 Hz male / 100–500 Hz female), falling back to `librosa.pyin`.

KEY NUMBERS

125.3	6.30	12,750
mean WPM (all)	mean syll/s	utterances used

F0 is dominated by gender, not region: female means run ~210–248 Hz, male ~141–174 Hz across every region — region effects must be read *within* a gender panel. Some region×gender cells (e.g. Mymensingh female, WPM std = 1204.6) show pathological variance, the symptom A7 was built to fix.

A7 VAD-based speaking rate (corrects A6)

`A7_vad_speaking_rate/analyze.py`

Purpose. Replaces A6's flawed energy-threshold trim with a trained voice-activity model, adds sanity guards against implausible utterances, and quantifies exactly how much the old method inflated WPM. The script frames itself as replacing the thesis's Table 4.4.

METHOD

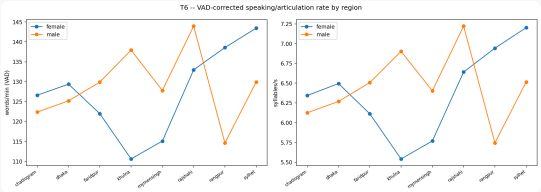
Speech duration via **Silero VAD** (threshold 0.5) instead of an energy trim. WPM/articulation rate computed on VAD duration; F0 also converted to **semitones re 100 Hz** for scale-invariant pitch comparison. Utterances are excluded from aggregates (but retained, tagged) if speech duration < 0.30s, speech ratio outside [0.20, 0.99], or WPM outside [60, 400]. Reads the verified, cleaned manifest and directly diffs its own numbers against A6's precomputed metrics per region×gender cell.

KEY NUMBERS

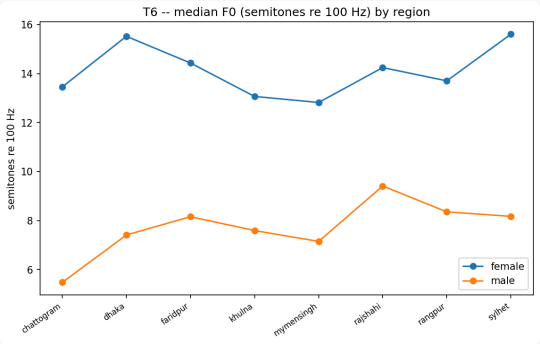
12,508	239	128.82
utterances used (of 12,747)	excluded (1.88%)	mean WPM (VAD-corrected)

−6.11%
overall WPM change vs. A6

The energy-trim baseline (A6) **overstated** mean WPM by roughly 6% corpus-wide once VAD and sanity guards replace it — per-cell corrections range from −1.0% (Chattogram female) to **−20.7%** (Khulna female). Most exclusions are tagged `bad_speech_ratio` (e.g. Khulna male: 66 utterances). Gender separation on the F0 semitone scale persists (female ~12.8–15.6 st vs. male ~5.5–9.4 st re 100 Hz).



VAD-corrected WPM and articulation rate by region, one line per gender.



Mean F0 (semitones re 100 Hz) by region, one line per gender.

A8 Audio quality & SNR

A8_audio_quality_snr/analyze.py

Purpose. Introduces a named, reproducible SNR estimator, tests whether SNR differs meaningfully by region, and tests whether noise quality — rather than accent — explains Part B's per-region WER differences.

METHOD

A **WADA-style SNR** estimator (Kim & Stern 2008 log-amplitude Gamma-shape statistic): solves for a Gamma shape parameter via `scipy.optimize.brentq`, then maps it to SNR(dB) through a locally-calibrated isotonic regression fit on synthetic mixtures built from 60 of this corpus's own clips plus additive white Gaussian noise at known SNR levels. Also computes a noise floor (median energy of the quietest 5% of 25ms frames) and keeps the old ad-hoc percentile-based estimator for comparison. Statistics: Kruskal–Wallis test across regions with epsilon-squared effect size; Spearman correlation between per-region median SNR and per-region Whisper WER.

KEY NUMBERS

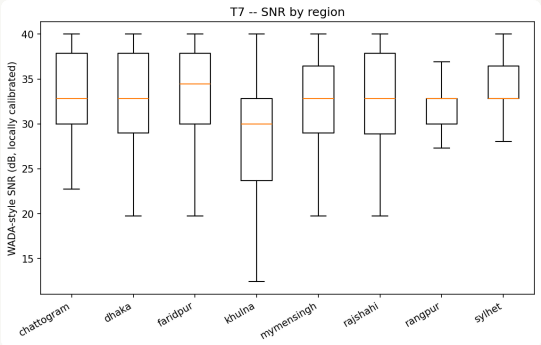
32.81 dB
overall median WADA-SNR

R^2=0.808
calibration fit quality

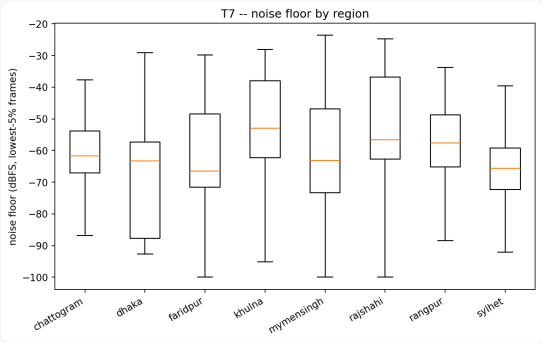
H=1081.1, p=3.6e-229
Kruskal–Wallis (region)

ε^2=0.084
effect size (small)

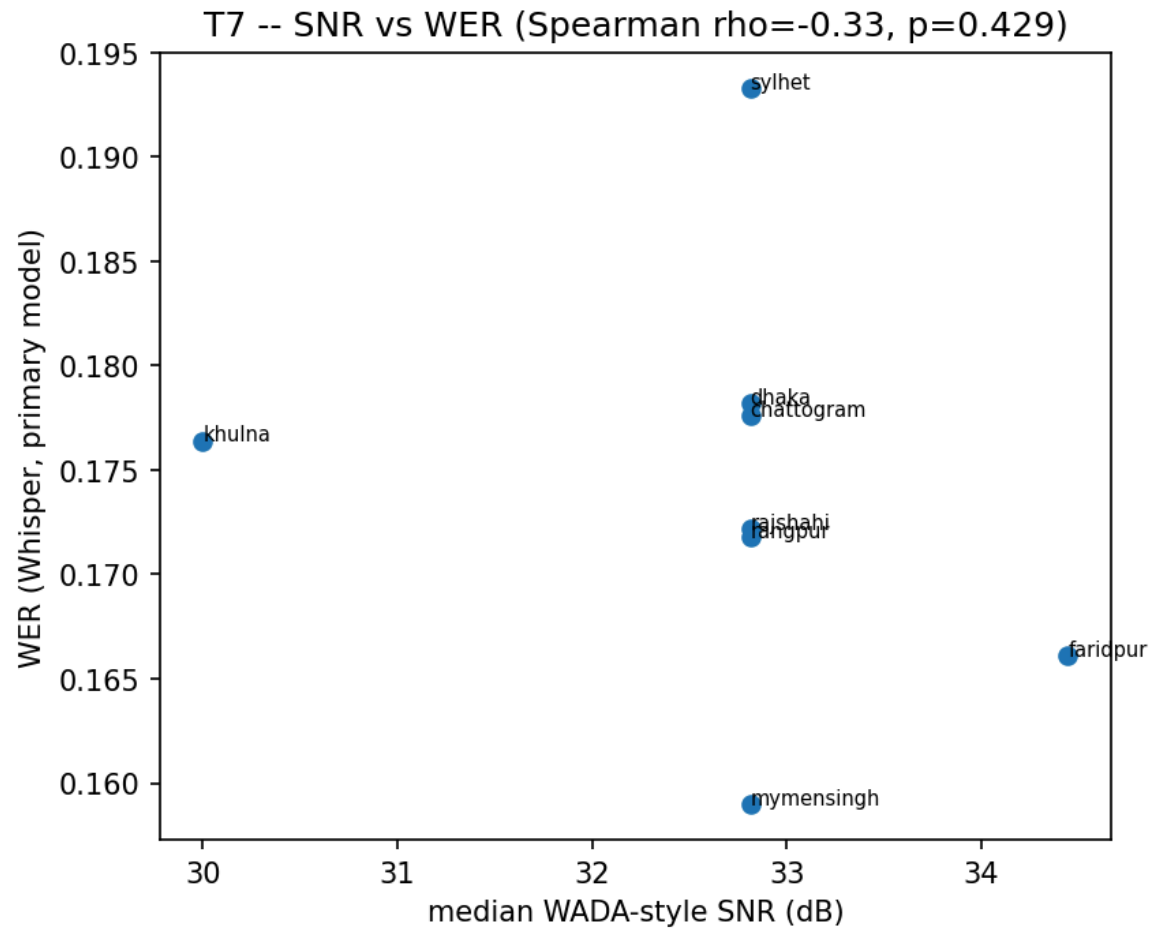
SNR differs significantly by region (Kruskal–Wallis) but the effect size is small — region only weakly determines SNR rank despite the large-n significance. App-recorded (female) regions run consistently lower SNR than lab-recorded (male) ones, e.g. Khulna: app 23.7 dB vs. lab 32.7 dB. Crucially, **`SNR does not explain WER`**: Spearman ρ = −0.327, p = 0.429 across the 8 regions — weak and non-significant, strengthening the case that Part B's per-region WER differences reflect accent, not audio quality.



WADA-style SNR (dB, locally calibrated) by region.



Noise floor (dBFS) by region.



Median WADA-SNR vs. Whisper WER per region — the figure that rules out (or flags) noise as the WER explanation.

PART B · ASR BENCHMARK & ALIGNMENT VERIFICATION

Speech-to-text benchmark & manifest correction

Two independent ASR models transcribe the whole corpus, both to benchmark recognition difficulty by region and — more importantly — to power an automated audit that checks every utterance's audio against *all 250* candidate sentences, catching mislabeling that a duration-based proxy alone would miss. This audit produces `verified_manifest.csv`, the corrected record every later Part reads from.

`package bpac_stt · cache Part_B/cache/predictions.csv`

B·1 wav2vec2-XLSR benchmark

`wav2vec2_stt/analyze.py`

Purpose. Fast CTC-based ASR benchmark, run first so results can be sanity-checked before paying for the slower Whisper pass.

METHOD

Model `shahruk10/wav2vec2-xls-r-300m-bengali-commonvoice` (CTC, XLS-R backbone), transcribes every utterance at 16 kHz mono via a Hugging Face ASR pipeline, checkpointing every 200 utterances into a shared prediction cache. Scoring: reference and hypothesis both NFC-normalized and punctuation-stripped identically, then scored with `jiwer` WER/CER; utterances flagged as empty/dead audio are excluded from the metric.

KEY NUMBERS

0.2542	0.0770	12,747
overall WER	overall CER	scored (of 12,750)

REGION	WER	CER	N
Sylhet `hardest`	0.2799	0.0931	2500
Khulna	0.2622	0.0817	1500
Dhaka	0.2566	0.0766	1500
Chattogram	0.2545	0.0736	1500
Rajshahi	0.2452	0.0725	1500
Rangpur	0.2454	0.0711	1250
Faridpur	0.2389	0.0681	1749
Mymensingh `easiest`	0.2314	0.0675	1248

B·2 Whisper-medium benchmark

`whisper_stt/analyze.py`

Purpose. Slower, stronger encoder-decoder benchmark using the same shared scoring pipeline.

METHOD

Model `bengaliAI/tugstugi_bengaliai-asr_whisper-medium` — a general (non-regional) Bengali Whisper, deliberately chosen so accent shows up as degradation rather than being absorbed by a model tuned to a specific region. Greedy decoding, 30-second chunking, forced Bangla language/task with a fallback ladder if the model's generation config rejects those kwargs. Same normalization + `jiwer` scoring as wav2vec2.

KEY NUMBERS

0.1760
overall WER

0.0701
overall CER

12,747
scored (of 12,750)

Whisper beats wav2vec2 on every region, both metrics — consistent with it being the larger, stronger model. Both models agree on the ranking's ends: **Sylhet is hardest, Mymensingh is easiest**, which is the region ranking the neural-embedding (Part C) and phonetic (Part D) analyses are asked to corroborate.

REGION	WER	CER	N
Sylhet `hardest`	0.1933	0.0840	2500
Dhaka	0.1782	0.0695	1500
Chattogram	0.1776	0.0696	1500
Khulna	0.1764	0.0699	1500
Rajshahi	0.1722	0.0673	1500
Rangpur	0.1718	0.0659	1250
Faridpur	0.1661	0.0638	1749
Mymensingh `easiest`	0.1590	0.0604	1248

B·3 Combined cross-model report

`create_report.py → combined_report/`

Purpose. Merge both models' predictions from the shared cache into one comparison: whole-corpus and per-region WER/CER matrices, plus a region-difficulty ranking averaged across models.

METHOD

Reads `cache/predictions.csv` directly (does not re-transcribe) and reruns the same scoring for every populated hypothesis column, producing a region × model CER/WER matrix with a `mean_across_models` ranking column.

âš **This report predates the alignment correction.** Its `summary.json` is timestamped before `verified_manifest.csv` existed, while the individual `wav2vec2_stt/` and `whisper_stt/` reports above were regenerated afterward — the discrepancy is visible directly in the numbers (e.g. this report's Rajshahi wav2vec2 CER is 0.0896 vs. the corrected 0.0725 shown in B·1). **The individual model reports above are the authoritative, current numbers**; this combined report needs a re-run against the current cache before its cross-model ranking should be cited.

B·4 Alignment verification

`alignment_verification/analyze.py · spotcheck.py · resolve.py`

Purpose. An ASR-based cross-check, independent of Part C's duration-based proxy, that every `sound_NNN.wav` actually contains the sentence its filename claims — and the source of the corrected, canonical manifest every later Part reads from.

METHOD — THE CER-AGAINST-ALL-250-CANDIDATES ALGORITHM

- Both ASR models' hypotheses are normalized (NFC + punctuation strip) alongside all 250 normalized script sentences.
- A **12,747 × 250 character-error-rate matrix** is built per model in one batched pass (`rapidfuzz.process.cdist` with Levenshtein distance, ~3.19M comparisons in seconds) — for every utterance, its hypothesis is scored against *every* candidate sentence, not just its labeled one.
- For each utterance: `best_match_sentence_id` = the candidate with minimum CER; `self_cer` = CER against its own labeled sentence.
- A misalignment is flagged when the **best match is a different sentence than the label** and the gap between self-CER and best-match CER exceeds 0.25 (`flag_offdiag`) — the decisive signal, since accent alone raises self-CER but leaves the correct sentence as best match. A softer `flag_highcer` (self-CER above the 99th percentile) and `flag_agree` (both models agree on a different best match) supplement it.
- Flagged and known-suspect utterances (carried over from Part C0's duration-deviation proxy) form a 300-row spot-check worklist, reviewed by ear via `spotcheck.py` and resolved into `verified_manifest.csv` by `resolve.py`.

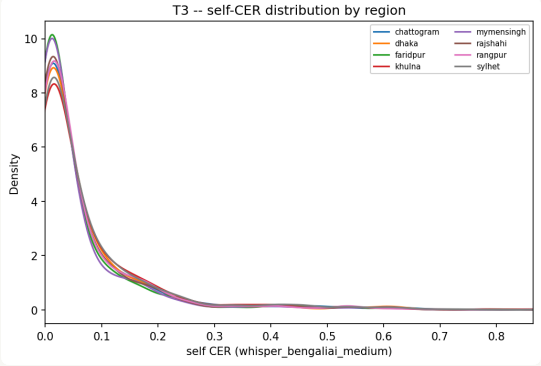
WHAT THE AUDIT FOUND

300	483	29	0
utterances flagged for review	reviewed as ok	corrected (one rotation bug)	excluded

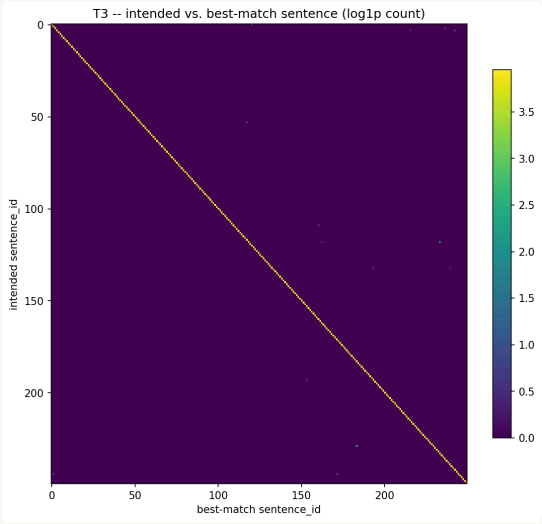
99.773%
alignment verification rate

The single systematic error found: **29 consecutive recordings from one Rajshahi speaker** (`rajshahi_male_002_fardin`, sound_080–sound_108) were labeled one sentence off — a sequential numbering rotation (sound_108 wraps back to sentence 80). Every one of the 29 was corrected in place (`sentence_id` fixed, audio untouched); nothing else in the corpus needed excluding. Final `verified_manifest.csv`: **`0 excluded, 0 needs-review`** across all 12,750 raw utterances.

Cross-validated against Part C0's independent duration-based suspect-cell detector: of 2,000 region×sentence cells, 85 were ASR-flagged and only 1 overlapped with the duration-based flags — the two detectors are largely complementary, and none of C0's duration-flagged "suspect sentences" showed ASR-confirmed misalignment on cross-check.



Self-CER density by region (primary model) — the accent-driven-degradation baseline the flags are measured against.



Intended sentence-id vs. ASR best-match sentence-id (log-scaled) — a clean diagonal is the visual evidence of correct alignment.

PART C · STEPS C0–C4

Neural-embedding accent structure

The central "is it accent, or is it just speaker identity?" question, asked with self-supervised speech embeddings. A first pass (v1, superseded) found that the tightest cluster in the embedding space coincided with the recording-format split rather than dialect — so this v2 pipeline first normalizes the corpus to remove that confound, then rebuilds the distance matrix and, decisively, tests whether region is predictable on *speakers the model has never seen*.

`package bpac_accent · models: content = WavLM-large (middle layer) · speaker = ECAPA-TDNN`

C0 Normalize corpus & diagnostics

`C0_Normalize_corpus/analyze.py`

Purpose. Strip the recording-format fingerprint (44.1/48 kHz, stereo/mono) that let v1's clustering pick up recording artifact instead of dialect, and run a duration-based proxy for sentence misalignment.

METHOD

Resamples every clip to **16 kHz mono** (optionally peak-normalized to −1.0 dBFS), drops empty/dead clips, and writes a checksummed clean tree plus a corpus-wide "checksum of checksums" for reproducibility. A separate diagnostic flags, per region, the fraction of the 250 shared sentences whose `sound_NNN.wav` duration deviates >40% from that sentence's cross-region median — a proxy for misalignment (later confirmed/refined by Part B's ASR-based audit).

KEY NUMBERS

12,750	3	12,747	0
input clips	dead / empty dropped	final clean corpus	excluded by verified manifest

Alignment-suspect rate by region: Dhaka 1.2% (3/250 sentences), Khulna 0.8%, Rajshahi & Rangpur 0.4%, all other regions 0% — low enough across the board that Part B's independent ASR-based audit was the deciding check (see B·4).

C1 Region accent-distance matrix

`C1_Region_accent_distance_matrix/analyze.py`

Purpose. Build an 8×8 region-pairwise "accent distance" on the clean corpus. With format now uniform, a surviving cluster reflects dialect, not a recording artifact.

METHOD

WavLM-large, **middle hidden layer** (chosen deliberately: the final layer drifts toward WavLM's pretraining objective — speaker identity and denoising — which suppressed accent and inflated speaker signal in v1's last-layer embeddings), mean-pooled over time. For each of the 250 shared sentences, the per-sentence mean embedding is subtracted across all speakers/regions (removing lexical content), distances are computed as standardized cosine distance, and averaged over every speaker pair sharing that sentence, then over all 250 sentences — a "parallel-aware" distance that exploits the corpus's core design property.

KEY NUMBERS

0.9832	1.0556
closest pair (Chattogram–Sylhet)	farthest pair (Rangpur–Sylhet)

Distance from Dhaka (descending): Sylhet 1.047, Mymensingh 1.040, Faridpur 1.017, Rangpur 1.013, Chattogram 1.012, Khulna 1.009, Rajshahi 0.990 — Sylhet sits furthest from the Dhaka standard on this measure, consistent with the ASR difficulty ranking in Part B.

C2 Speaker vs. accent separability

`C2_Speaker_vs_accent_embedding_separability/analyze.py`

Purpose. The confound test: does the content embedding separate by *region*, or is it really just separating by *speaker identity*? Leave-one-speaker-out (LOSO) classification is the decisive version of this test — it can only succeed by generalizing to voices the classifier has never heard.

METHOD

Silhouette scores compare three spaces (ECAPA speaker embedding, WavLM content embedding, and WavLM with each speaker's own centroid subtracted). The decisive test: for each of the 51 speakers held out in turn, an LDA classifier is fit on the *other* 50 speakers' WavLM embeddings to predict region, then evaluated on the held-out speaker — repeated across all 51, chance = 1/8 = 0.125.

KEY NUMBERS

0.251	2.0×	0.675
LOSO accuracy (unseen speakers)	lift vs. chance (0.125)	Sylhet recall (most distinct)

Region is predictable on **unseen** speakers at roughly twice chance — by the project's own threshold (≥2× chance = genuine, 1.3–2× = real but modest, ≈chance = confound-dominated), this reads as a genuine accent signal, not speaker memorization. Sylhet is by far the most distinguishable region (67.5% recall); the biggest confusions run Dhaka↔Rajshahi and Rangpur↔Khulna. *But* this pooled number is exactly what Part C3 goes on to stress-test against the gender/pipeline confound.

C3 LOSO within gender — the confound defense

`C3_LOSO_within_gender/analyze.py`

Purpose. Recording pipeline is *perfectly* confounded with gender (lab = male, app = female) — not a tendency, a 1:1 mapping. So C2's pooled LOSO result could in principle be detecting recording channel rather than accent. C3 reruns LOSO *separately within each gender*, which holds recording channel constant, plus a permutation test and bootstrap confidence interval. The report itself calls this **"the single most important defensive result in the paper."**

METHOD

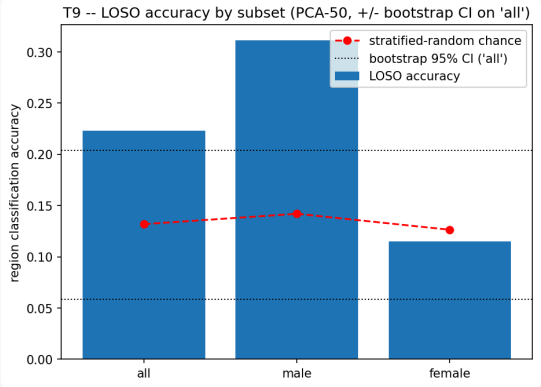
The same PCA(50)+LDA LOSO classifier as C2, run independently on three subsets — `all` (pooled), `male` (= lab-recorded), `female` (= app-recorded) — compared against a *stratified-random* chance baseline (accounts for regions' unequal sizes, stricter than uniform 1/8 chance). A speaker-level permutation test (100 shuffles, whole speakers moved together to preserve the nested speaker-in-region structure) and a speaker-resampled bootstrap (100 resamples) add statistical backing. An ECAPA (speaker-identity) control checks whether a model that shouldn't know accent can predict region about as well — if so, the signal would be speaker leakage, not accent.

HEADLINE RESULT: LOSO ACCURACY BY SUBSET

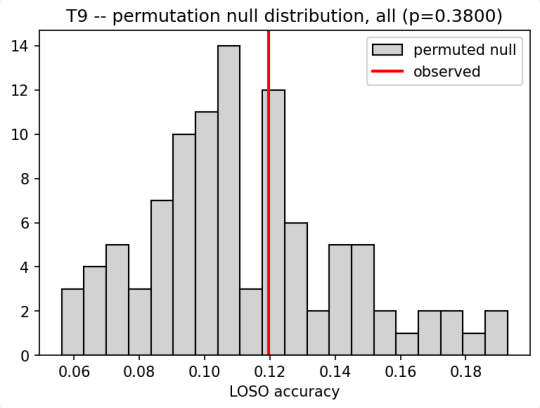
SUBSET	SPEAKERS	ACCURACY	STRATIFIED CHANCE	LIFT
All (pooled)	51	0.2233	0.1319	1.69×
Male (= lab)	26	**0.3111**	0.1420	**2.19×**
Female (= app)	25	0.1153	0.1264	0.91× (below chance)

Asymmetric result. Within male/lab-recorded speakers, LOSO accuracy is strong and — confirmed by permutation testing — **`highly significant (p = 0.0000`**, 0 of 100 speaker-shuffled permutations reached the observed accuracy; bootstrap 95% CI [0.077, 0.360] excludes zero-lift**)**: region is genuinely predictable on unseen lab-recorded speakers, so that half of the corpus carries a real accent signal, not a channel artifact. Within female/app-recorded speakers, accuracy sits at or below chance. The pooled number, tested the same way, is **not** statistically significant on its own (permutation p = 0.38) — so the honest headline is not "accent is detectable at 2× chance," it is **"accent is detectable, and the detectable signal is concentrated in the lab-recorded/male half of the corpus."** The female/app gap is reported as an open question (compression or mic variability in the app pipeline, or a genuinely harder detection problem for those regions/speakers) — not swept into the average.

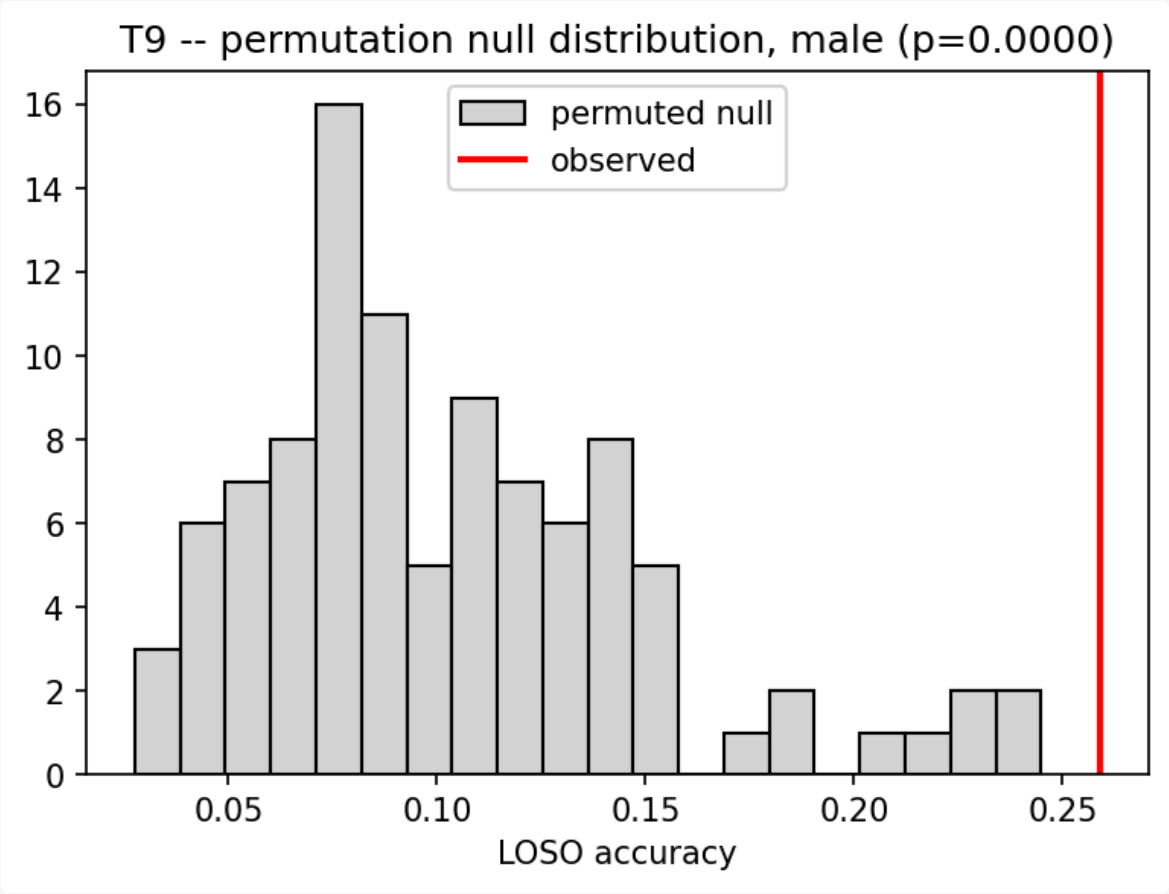
The ECAPA speaker-identity control predicts region far more weakly (1.28× lift) than the content embedding (1.69–2.19×), supporting that WavLM's signal is not simply speaker leakage.



LOSO accuracy per subset vs. stratified-random chance, with bootstrap 95% CI for the pooled subset.



Permutation null distribution (pooled, n=100) vs. observed accuracy — p = 0.38, not significant.



Permutation null distribution for the male/lab subset (n=100) — the observed accuracy (0.259 in this reduced-dimension test setting) sits entirely outside the null, p = 0.0000.

C4 Distance-matrix statistics

`C4_distance_statistics/analyze.py`

Purpose. Adds statistical rigor to C1's raw distance matrix: is the region ordering reliable, or noise? Does acoustic distance track geography? How faithfully does a dendrogram summarize the full matrix?

METHOD

Bootstrap confidence intervals per region pair (resampling speakers with replacement *within region*, 200 iterations, rebuilding C1's distance matrix each time). A Mantel test correlates the acoustic distance matrix against a geographic (haversine) distance matrix built from each region's district coordinates, permutation-tested 10,000 times. Ward hierarchical clustering plus cophenetic correlation measures how well a dendrogram reproduces the full matrix.

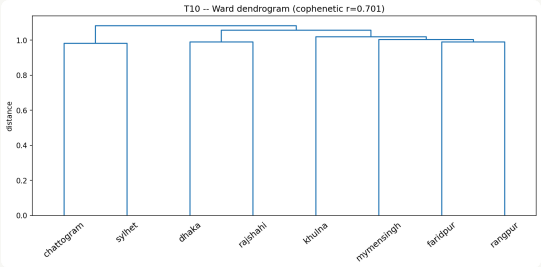
KEY NUMBERS

CIs don't overlap
closest vs. farthest pair

r=0.175, p=0.33
Mantel (Pearson, vs. geography)

0.7006
cophenetic correlation

The closest pair (Chattogram–Sylhet, CI [0.962, 1.020]) and farthest pair (Rangpur–Sylhet, CI [1.024, 1.126]) have non-overlapping confidence intervals, so this ordering is statistically reliable. But acoustic distance does **not** significantly track geographic distance between regions (Mantel p = 0.33 Pearson, 0.42 Spearman) — a null result the report treats as expected given how weak the unsupervised distances are overall, and cites as motivation for C3's discriminative LOSO test and Part D's phonetic distance matrix as the more decisive analyses.



Ward dendrogram of the 8 regions by embedding accent-distance (cophenetic r = 0.70).



Region × region accent-distance heatmap with bootstrap-backed pairwise values.

PART D · STEPS D0–D7

Classical acoustic-phonetic map

Where Part C shows an accent signal *exists* in pooled neural embeddings, Part D shows *what it is made of*: phone durations, rhythm typology, vowel space, sibilant spectra, stop VOT, and intonation — all measured directly off forced-aligned phone boundaries and compared across the 8 regions because every speaker read the same 250 sentences. Entirely CPU-bound; no GPU required.

`package bpac_phon · forced aligner: Montreal Forced Aligner (MFA), trained from scratch on this corpus`

D0 Pronunciation lexicon

`D0_lexicon/analyze.py`

Purpose. Build the Bangla pronunciation dictionary the forced aligner needs, since off-the-shelf options weren't usable here.

METHOD

Every word type in the 250-sentence script is run through a **rule-based grapheme-to-phoneme (G2P) converter** written for this project — `epitran` was uninstallable under this repo's Python version and `bnlp_toolkit` has no G2P module at all. Key rule decisions: à¦¶/à¦·/à¦¸ all collapse to one canonical phone `s` (the regional /s/→/ʃ/ shift is deliberately *not* baked into the lexicon, so it remains something D4 can measure acoustically rather than assume); word-final bare consonants drop their inherent vowel, all other bare consonants keep it (flagged as the hardest rule to get right). The top 500 most frequent words plus a random 200-word tail (700 words total) were manually reviewed by the corpus author.

KEY NUMBERS

2,011	100%	45	700 / 700
unique word types	lexicon coverage	phone inventory size	words manually reviewed

Manual review verdict: **accurate overall**, no per-word corrections needed. 1,093 words flagged as carrying schwa-deletion risk and 517 as containing consonant conjuncts — tracked for transparency even though no changes were required.

D1 Forced alignment + held-out-region check

`D1_alignment/prepare_corpus.py · finalize.py · prepare_heldout_split.py · heldout_align_and_compare.py`

Purpose. Produce phone- and word-level time boundaries for every utterance — the foundational data every later D-step reads — using an acoustic model trained from scratch on this corpus, plus a robustness check that the aligner learned general Bangla phonetics rather than memorizing regional pronunciation.

METHOD

The Montreal Forced Aligner (MFA) is trained on the full corpus using D0's lexicon, then used to force-align every utterance into phone/word TextGrids. The robustness check retrains a *second* acoustic model on the 41 non-Sylhet speakers only, then uses it to align the 10 held-out Sylhet speakers — if the resulting phone boundaries closely match the full-corpus model's Sylhet alignments, the aligner isn't just memorizing Sylhet-specific pronunciation, and the phonetic measurements built on top of it (D2–D7) are trustworthy rather than circular.

KEY NUMBERS

12,635 / 12,747	959,881	170,033	2.33%
utterances aligned	phone tokens	word tokens	phones under 10ms

Held-out-region check: PASS. Across 182,374 matched phone tokens, the held-out model agreed with the full-corpus model on **82.2% of boundaries within 10 ms**, 94.4% within 20 ms, and 96.5% within 30 ms (median boundary difference: 0.0 ms). This is the evidence that the phonetic measurements built on these alignments in D2–D7 are trustworthy rather than circular.

D2 Phone duration & rhythm typology

`D2_duration_rhythm/analyze.py`

Purpose. Two things: (1) use the aligner's own fit quality as a free "divergence from standard Bangla" signal per region, and (2) measure standard rhythm-typology metrics (%V, ΔC, nPVI) that quantify a language variety's timing character.

METHOD

A divergence score combines each region's rate of suspiciously-short phones and suspiciously-long non-vowels into a z-score sum. Rhythm metrics follow the standard

Ramus/Grabe–Low battery: phones are collapsed into vocalic (V) and consonantal (C) interval runs per utterance, then **%V** (vocalic proportion of total duration), **ΔC** (standard deviation of consonantal-interval duration), and **nPVI** (normalized pairwise variability index across consecutive vocalic intervals) are computed. Per-phone duration shifts are tested with a mixed-effects model (`duration ~ region + (1|speaker)`), FDR-corrected across 39 phones with ≥200 tokens each.

KEY NUMBERS

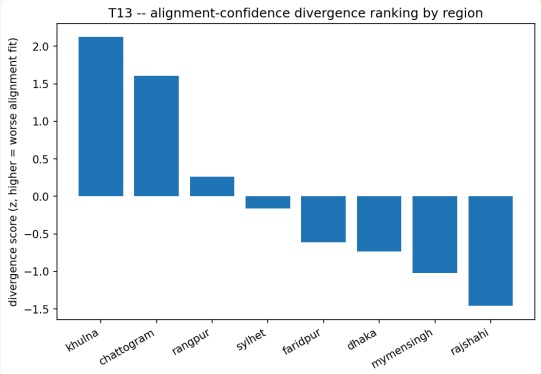
Khulna
highest divergence (2.12)

Rajshahi
lowest divergence (−1.46)

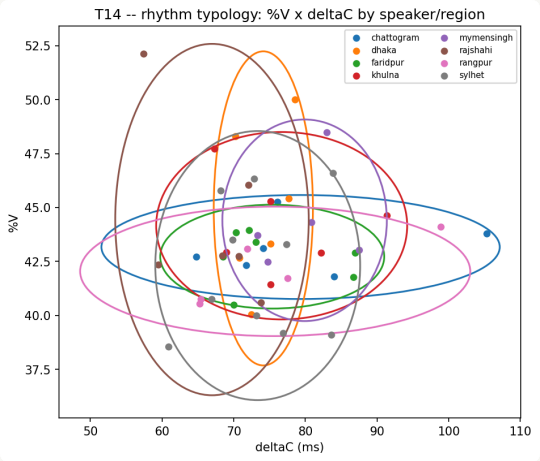
42.1–45.0%
%V range across regions

67–80 ms
ΔC range across regions

Sylhet is **not** the most divergent region by alignment-fit quality — Khulna and Chattogram score higher. Divergence doesn't significantly track Part B's WER ranking (Spearman ρ = 0.31, p = 0.46, n=8 regions). Rajshahi stands out on rhythm timing with the tightest consonantal-duration variability (ΔC = 67ms vs. up to 80ms elsewhere) — this consonant/rhythm-timing axis, not the Sylheti sibilant shift, turns out to be the dominant phonetic signal once everything is fused in D6.



Alignment-fit divergence score by region (higher = further from the aligner's learned norm).



%V × ΔC rhythm-typology scatter, per speaker, colored/ellipsed by region.

D3 Vowel space

D3_vowel_space/analyze.py

Purpose. Characterize each region's vowel quadrilateral, controlling for vocal-tract-length differences between speakers — necessary because Part C already showed raw acoustic geometry is dominated by speaker identity, not accent.

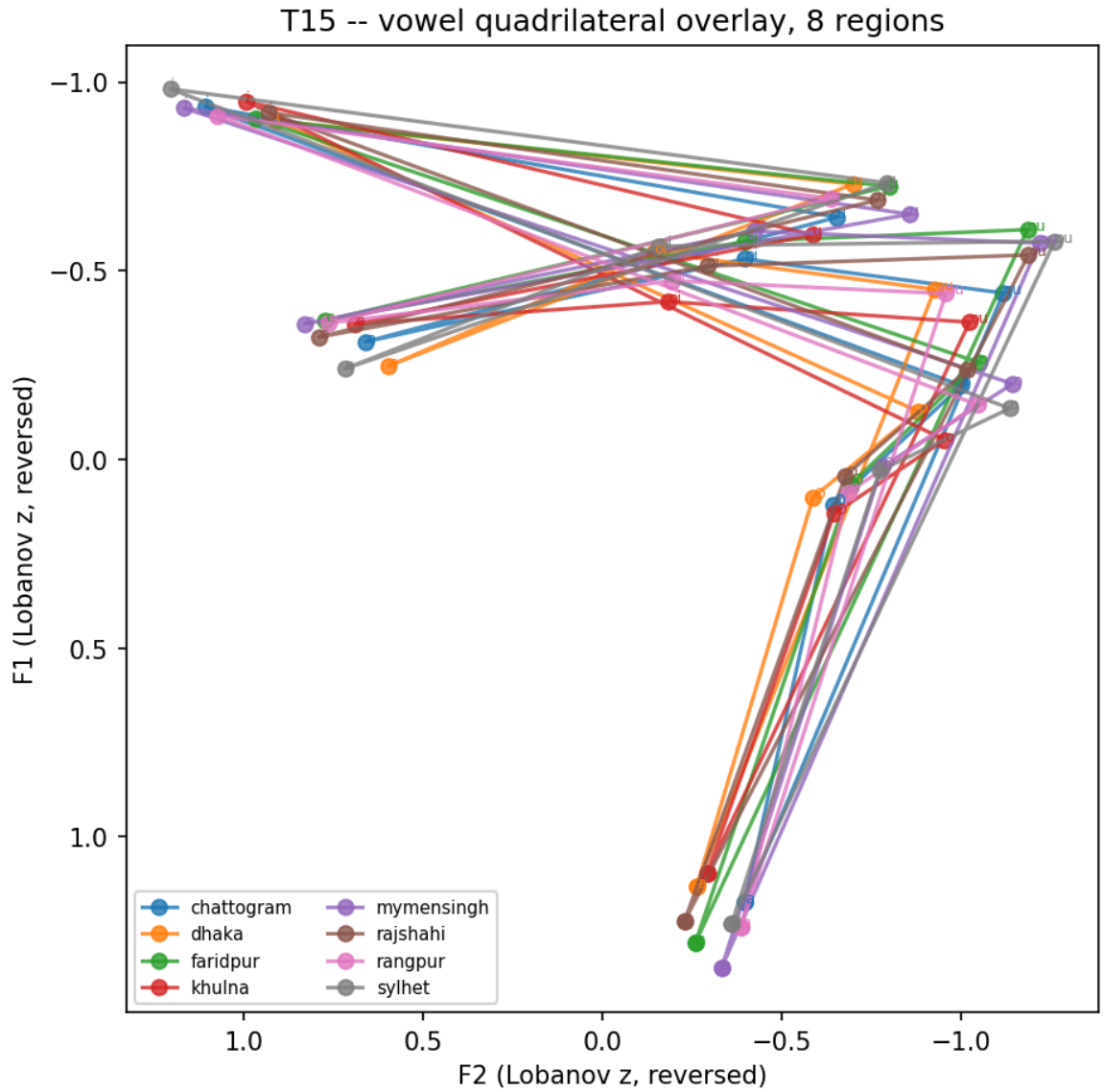
METHOD

Formants (F1/F2/F3) are extracted via Praat's Burg algorithm at the midpoint of every vowel token ≥50 ms. **Lobanov normalization** then z-scores each formant within-speaker (`(F − speaker_mean) / speaker_std`), which removes vocal-tract-length differences while preserving each speaker's relative vowel-space geometry. Vowel-space area is the convex-hull area over each region's per-vowel centroids; **Pillai scores** (MANOVA Pillai's trace) quantify how acoustically merged or distinct near-vowel pairs are (e-i, o-u, O-o, a-O).

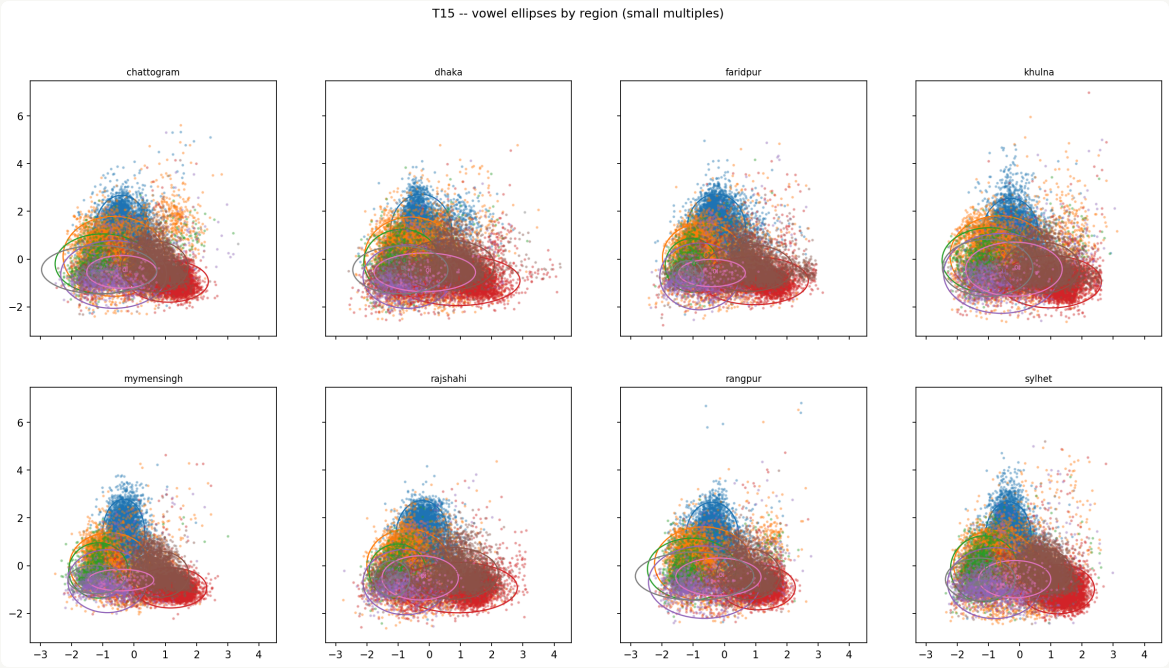
KEY NUMBERS

203,425	Sylhet	Khulna
vowel tokens retained (98.9%)	largest vowel space (2.64)	smallest vowel space (1.92)

Pillai scores for the e–i vowel pair range from 0.226 (Faridpur, most merged) to 0.403 (Sylhet, most distinct) — showing regions genuinely differ in how separated their vowel categories are, not just in overall space size.



8-region overlay of Lobanov-normalized vowel quadrilaterals (F2 vs. F1, IPA-convention axes) — the main vowel-space figure.



95% confidence ellipses for each vowel's token cloud, one panel per region.

D4 Consonants — sibilants & VOT

`D4_consonants/analyze.py`

Purpose. The analysis motivating the whole project: put an actual acoustic number on the sibilant spectral shift often cited anecdotally for Sylheti speech, plus measure voice-onset-time (VOT) for the four-way stop series.

METHOD

Unlike every other D-step, this one reads **raw, full-bandwidth audio** (44.1/48 kHz, resampled to 32 kHz) rather than the 16 kHz normalized corpus — at 16 kHz the Nyquist limit would truncate exactly the upper-frequency energy the /s/ vs. /ʃ/ spectral distinction depends on. For every sibilant token, **spectral center of gravity (CoG)** is computed on the middle 50% of the segment (Hamming-windowed, pre-emphasized, restricted to the 500–8000 Hz band) — the acoustic correlate of a sibilant's perceived "sharpness." VOT is estimated semi-automatically (burst detection via high-frequency energy, voicing onset via autocorrelation periodicity) and explicitly flagged provisional pending manual verification.

KEY NUMBERS

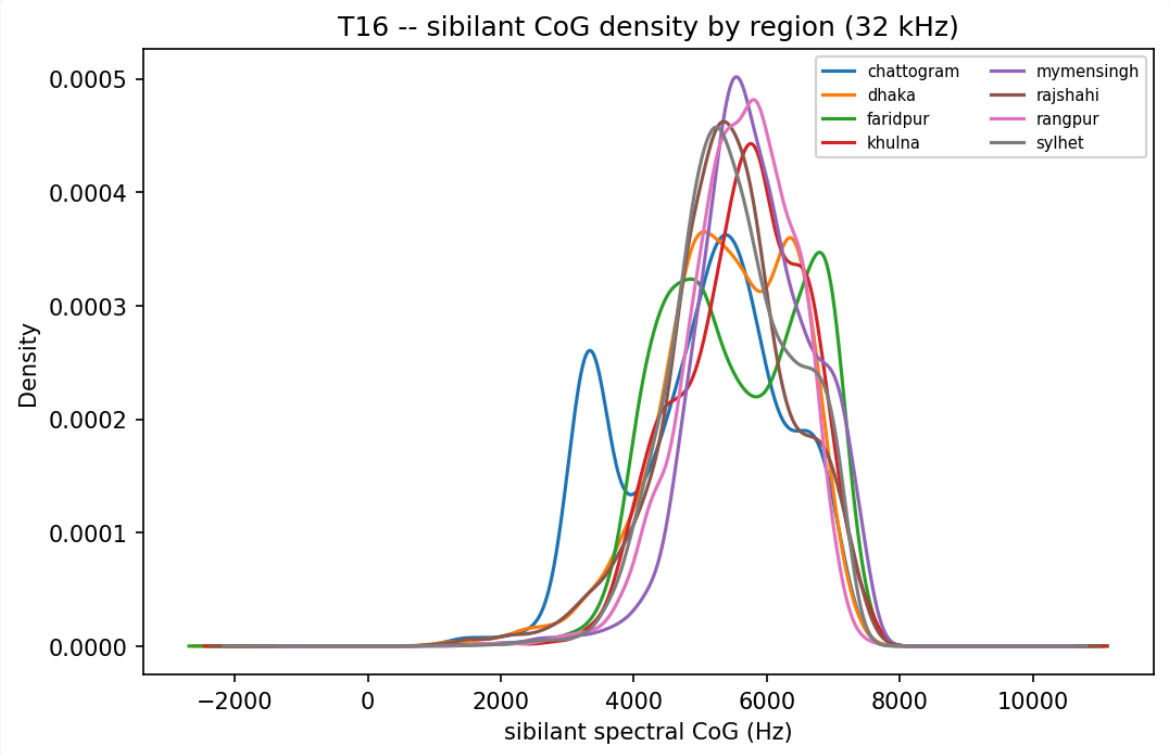
51,059
sibilant tokens measured

Mymensingh
highest CoG (5,812 Hz)

Chattogram
lowest CoG (5,001 Hz)

+28.7 Hz
Sylhet's shift from corpus mean

Measured directly, the Sylhet sibilant shift is **small** — only +28.7 Hz (0.029 SD) from the corpus mean, and Sylhet is not even the highest-CoG region (Mymensingh is). This is the acoustic result behind the thesis's headline finding: the anecdotal Sylheti sibilant shift, traced back to its usual literature source, turns out not to be acoustically measured or localized to Sylhet there either, and is a near-null effect once measured directly here. VOT patterns look phonetically sane (short-lag for voiced unaspirated stops, long-lag for aspirated ones) but remain provisional — manual verification of the ~300-token check worklist was never completed, so D6 excludes VOT from the fused distance matrix on that basis.



Sibilant spectral center-of-gravity density by region — likely the most-quoted figure in the paper.

D5 Intonation

D5_intonation/analyze.py

Purpose. Show what the parallel-sentence design uniquely makes possible: eight region-average F0 contours *for the same sentence*, directly comparable because every region read identical text.

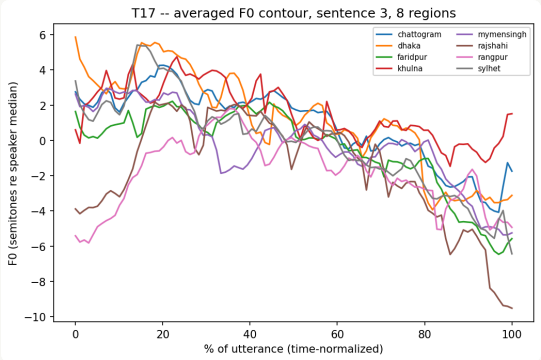
METHOD

F0 is extracted per utterance, converted to **semitones relative to each speaker's own median F0** (removing pitch-range and sex differences while preserving contour shape), then linearly time-normalized onto 100 points. Summary features (F0 range, onset, final slope, peak count) are computed per region, plus an 8×8 contour-distance matrix (mean absolute difference between region-average curves, averaged across all shared sentences). The six sentences with the highest cross-region divergence are plotted individually.

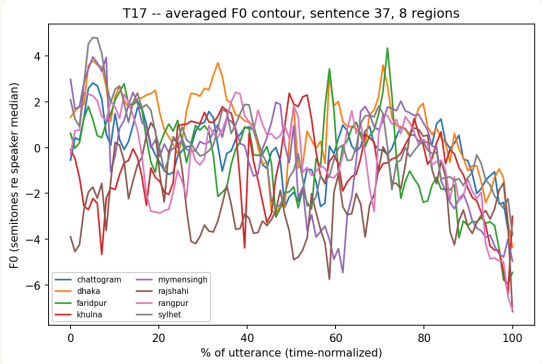
KEY NUMBERS

99.98%	15.97–19.12 st	Khulna
utterances with usable contour	F0 range across regions	most distant from all other regions

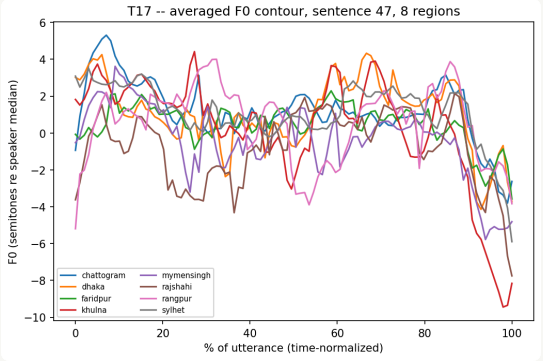
Closest contour pair: Sylhet–Chattogram; every region's contour ends in a fall (final slope negative throughout), steepest in Faridpur. All figures below plot the same eight region-average curves for one sentence at a time.



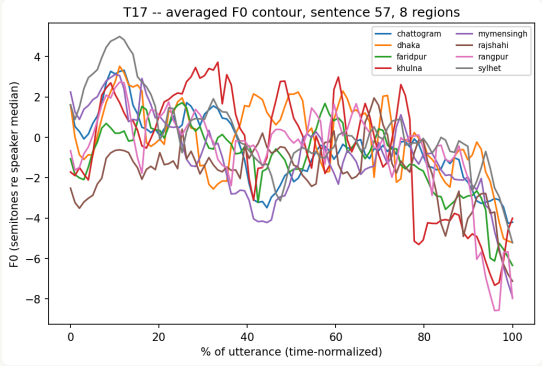
Sentence 3 — F0 contour (semitones vs. % time), 8 regions overlaid.



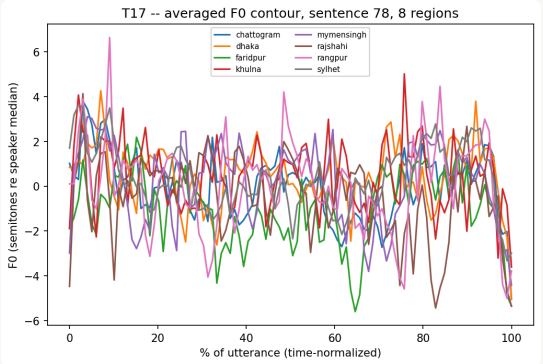
Sentence 37 — F0 contour, 8 regions overlaid.



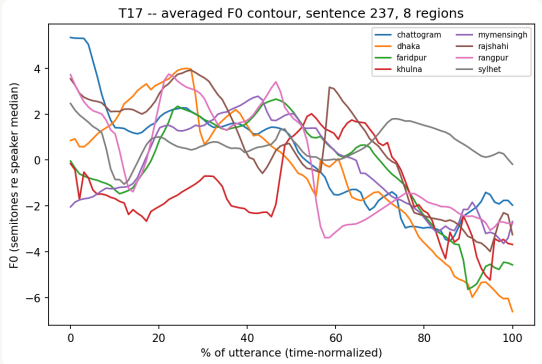
Sentence 47 — F0 contour, 8 regions overlaid.



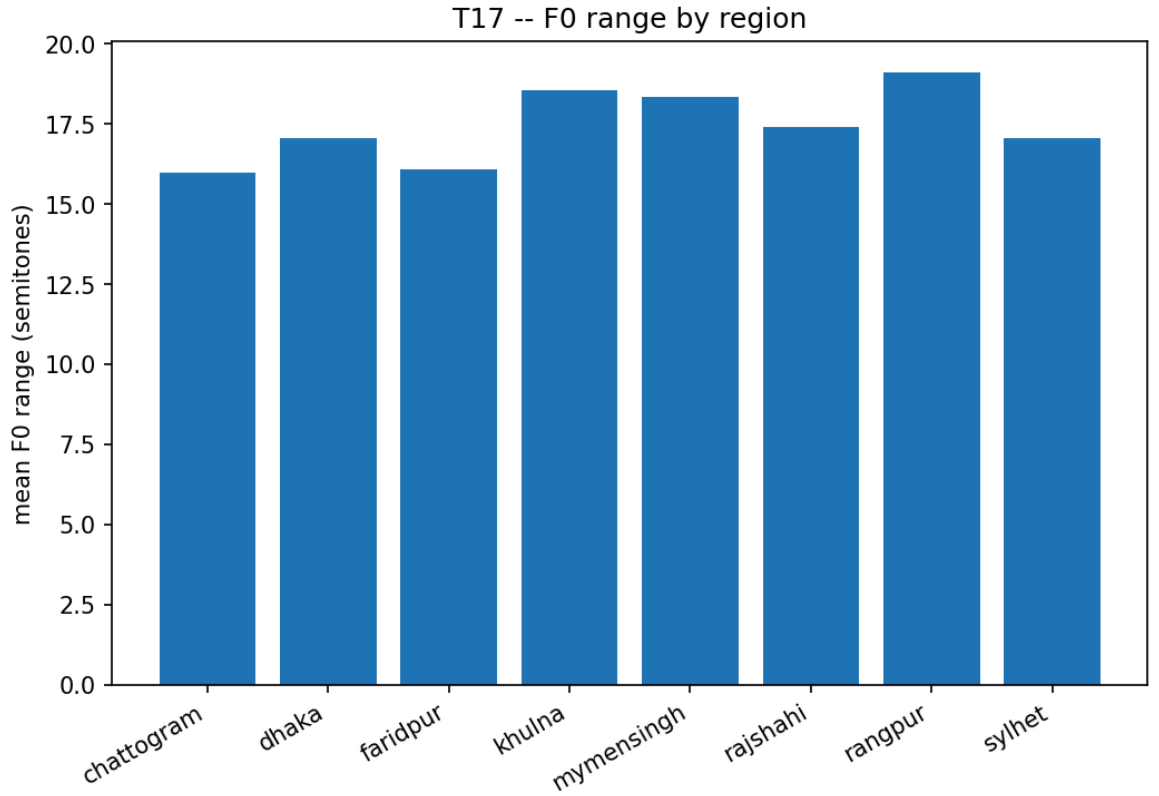
Sentence 57 — F0 contour, 8 regions overlaid.



Sentence 78 — F0 contour, 8 regions overlaid.



Sentence 237 — F0 contour, 8 regions overlaid.



Mean F0 range (semitones) by region.

D6 Fused phonetic distance matrix

`D6_phonetic_distance/analyze.py`

Purpose. Combine D2–D5's separate measurements into one interpretable 8×8 region distance matrix — the classical-phonetics counterpart to Part C's neural-embedding distance matrix, and the input D7 tests for convergence.

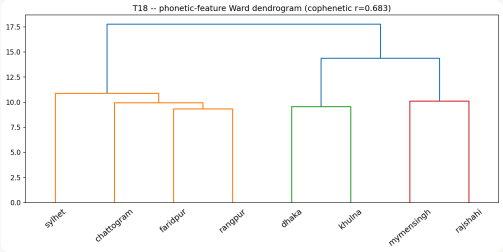
METHOD

Seven standardized feature blocks — phone duration (40 features), rhythm (7), vowel formants (16), vowel-space area (1), Pillai scores (4), sibilant CoG (1), F0 summary (5) — are concatenated into a 74-feature matrix across the 8 regions (VOT excluded on the policy grounds noted in D4). Euclidean distance plus Ward hierarchical clustering produce the matrix and dendrogram; cophenetic correlation checks how faithfully the tree represents the full matrix; a feature-block sensitivity analysis (drop one block, recompute, correlate with the full result) identifies which measurement actually drives the distances.

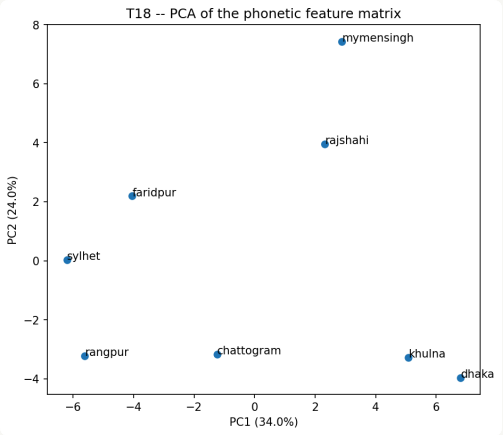
KEY NUMBERS

0.6826	34.0%	82.9%
cophenetic correlation	variance on PC1	Dhaka+Khulna branch support

The feature-block sensitivity analysis is the thesis's central phonetic finding: dropping **phone duration** collapses the fused matrix's correlation with the full result to **0.511** — by far the most influential block. Dropping **sibilant CoG** — the feature block conceptually central to the project's original motivating question — changes almost nothing (correlation 1.000). **Consonant/rhythm timing, not the Sylheti sibilant shift, is the dominant phonetic axis of regional variation** once everything is measured and fused together.



Ward dendrogram of the fused phonetic distance matrix (cophenetic r = 0.68).



PCA of the region × feature matrix — PC1 explains 34% of variance.

D7 Convergence test

`D7_convergence/analyze.py`

Purpose. The paper's headline cross-validation: do two completely independently-derived 8×8 region distance matrices — Part C's opaque neural-embedding distances and Part D's hand-measured phonetic distances — actually agree?

METHOD

A **Mantel test** (10,000 permutations, Pearson and Spearman variants) correlates the two distance matrices directly. Separately, both matrices are reduced to 2D via multidimensional scaling and Procrustes-aligned, so any geometric agreement can be visualized as connector lines between each region's two positions. A per-region check correlates each region's distance *profile* (its row in the matrix, i.e. its distance to every

other region) between the two matrices — a less demanding, per-region version of the same question.

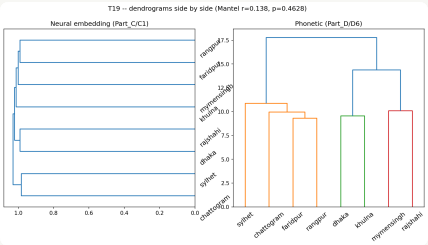
KEY NUMBERS

r=0.138, p=0.46
Mantel test (Pearson)

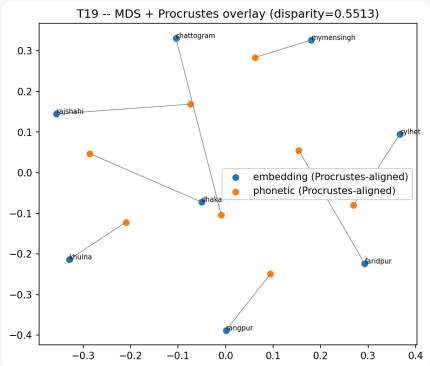
r=0.226, p=0.23
Mantel test (Spearman)

0.90–0.96
per-region row correlation range

The whole-matrix Mantel test is **not statistically significant** in either variant — at the level of the full 8×8 matrix, the two independently-derived views of regional structure do not converge with the strength the framing might suggest. But per-region row correlations are consistently high (0.90–0.96 across all 8 regions), suggesting real partial agreement that a test with only 8 regions (28 pairs) may simply be underpowered to confirm at the whole-matrix level. Neither matrix is a proxy for raw geography either (Mantel vs. geographic distance: r = 0.175 embedding, r = −0.197 phonetic, both non-significant) — both are measuring something other than which regions are physically closer together.



Neural-embedding (left) vs. phonetic (right) dendrograms, side by side.



2D MDS of both matrices, Procrustes-aligned — connector lines show each region's embedding-position vs. phonetic-position.

PART E · VALIDATION

Signal-processing validation

There is no human listening study anywhere in this repository — regional-origin and accent labels are validated at recruitment, not by post-hoc listener judgment. Part E's single check is a signal-processing sanity test that answers a specific reviewer objection: the thesis's voice-cloning models were trained on a *denoised* variant of BPAC, so why does this paper's phonetic analysis deliberately *not* denoise?

`package bpac_valid`

E1 Denoise check

`E1_denoise_check/analyze.py`

Purpose. Spectral-gating denoisers suppress broadband noise-like energy — which is exactly what fricatives and aspiration bursts are made of. This check demonstrates that denoising would attenuate the very acoustic cues Part D's sibilant-CoG and VOT measurements depend on, settling the "why didn't you denoise?" question with one paired comparison.

METHOD

2,000 sibilant tokens are stratified-sampled (by region × phone) from Part D4's output, and for each, the matching raw-audio segment is re-extracted from the clean corpus. Each segment is run through `noisereduce`'s spectral-gating denoiser (library defaults — the original thesis's exact parameters were not recoverable, and this is stated explicitly in the report). Spectral center-of-gravity (CoG) is computed on both the clean and denoised segment using the *same measurement function Part D4 uses for accent analysis* — so the comparison is apples-to-apples with the real analysis, not a synthetic proxy. A paired t-test, Wilcoxon signed-rank test, and Cohen's d quantify the shift; separately, the Sylhet-vs-rest sibilant-CoG effect size (Cohen's d) is computed once on clean and once on denoised audio to show directly how much denoising would shrink the group-separation signal the accent analysis depends on.

KEY NUMBERS

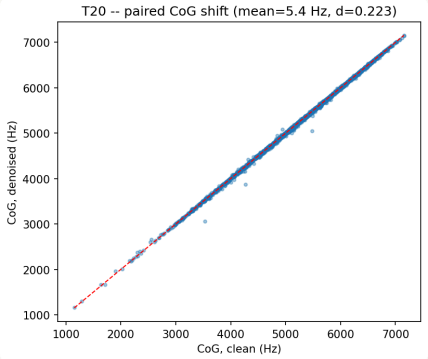
2,000
sibilant tokens tested

5.4 Hz
mean CoG shift after denoising

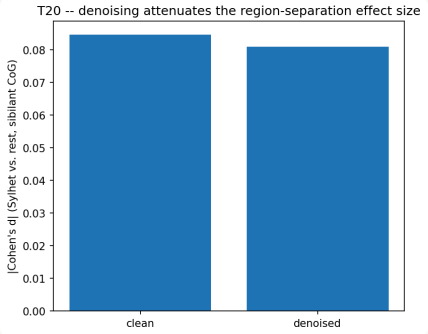
d = 0.223
paired Cohen's d

4.3%
region-separation attenuation

Denoising produces a statistically significant ($p < 10^{-22}$, both paired t-test and Wilcoxon — unsurprising given n=2,000) but **small** mean CoG shift (Cohen's d ≈ 0.22), and shrinks the Sylhet-vs-rest region-separation effect by only **4.3%** (Cohen's d: −0.0846 clean → −0.0810 denoised). The report frames this as a balanced result, not a one-sided argument: the effect is real and directionally supports not denoising for phonetic measurement, but it's modest — so using denoised audio for the separate voice-cloning work remains reasonable.



Center-of-gravity, clean vs. denoised, per sibilant token — points near the y=x line show minimal shift.



|Cohen's d| for the Sylhet-vs-rest sibilant-CoG separation, clean vs. denoised — "the argument in one number."

SHARED INFRASTRUCTURE & TOP-LEVEL UTILITIES

What ties the five Parts together

Code that every Part depends on but that isn't itself one of the five research questions: a single shared manifest-builder, a corpus-wide consistency audit, and the utilities that build the public dataset release and run standalone data-quality scans.

`common/` Shared manifest, provenance & consistency audit

Purpose. Dependency-light (stdlib + pandas only) shared code used by every Part's own package, so the manifest schema and summary-report contract can't silently drift between Parts A–E.

METHOD

- **`manifest_core.py`** — the single implementation every Part's manifest-builder wraps. Walks the region/speaker/sentence directory tree, and derives `pipeline` from `gender` as a hard-coded, asserted invariant (`lab = male, app = female`) — a recruitment fact, not a measurement.
- **`provenance.py`** — every task's `summary.json` is built from a shared helper guaranteeing it always carries `task_id, generated_at, config_hash, n_utterances_in, n_utterances_used`, so every report in this document is traceable to an exact config version.
- **`consistency_check.py`** (wired into `make check`) — walks every `reports/summary.json` in the repository and reconciles its input count against the corpus's canonical sizes (12,750 raw / 12,747 clean), flagging any unexplained row-count drop between stages as a hard failure.

CORPUS SCHEMA, VERIFIED

12,750	14
utterances documented	manifest columns

true
pipeline/gender confound holds corpus-wide

Pipeline×gender counts: `app/female = 6,250`, `lab/male = 6,500` — confirming the confound is exact, not approximate. Raw sample rates found in the corpus: 44,100 Hz and 48,000 Hz; channels: mono and stereo — exactly the format variation Part C0 normalizes away.

`root/` Standalone QA utilities & the public release builder

`check_audio_quality.py` — an independent per-file audio-quality scan (sample rate, channels, RMS, estimated SNR, clipping, silence) across all 12,750 files. All 8 regions get an automatic "suspect" flag, but purely because of the corpus-wide mixed 44.1/48 kHz and mono/stereo formats — every robust z-score for duration/RMS/SNR/silence/clipping stays well under the significance threshold. No individual region stands out as an outlier once format variation is accounted for.

`mark_empty_files.py` — scans for dead/silent recordings. Found **3 empty files out of 12,750** (0.02%): 2 in Mymensingh, 1 in Faridpur, zero elsewhere — these are the same 3 clips Part C0 drops when building the clean corpus.

`build_hf_release.py` — builds the anonymized, publication-ready copy of the corpus for Hugging Face. Reads `verified_manifest.csv` (and asserts it has 0 excluded / 0 needs-review rows — i.e. it can only run against the fully-reviewed manifest), strips every participant name from the identifiers, and re-derives collision-free anonymous speaker codes in the form `<region>_<gender>_<seq>`.

Reading order. Part A establishes that the corpus is what it claims to be (fully parallel, balanced, audio quality doesn't confound the rest). Part B benchmarks ASR difficulty by region and, along the way, corrects a manifest labeling error, producing `verified_manifest.csv`, which every later step reads. Part C asks whether region is detectable in self-supervised embeddings at all — and, critically, whether that signal survives holding speakers out entirely, split by the one hard confound (recording pipeline = gender). Part D independently asks what regional accent is *made of* in classical phonetic terms, then checks whether that hand-built picture agrees with Part C's neural one. Part E closes a specific methods question (why the corpus isn't denoised) that only makes sense once Part D's measurements exist.

Every number, table, and figure above is quoted directly from the `reports/` output committed in this repository — run `make all` to regenerate everything from scratch, or see each Part's own `README.md` for the individual commands.